SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

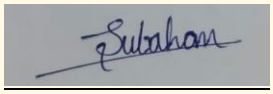
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	T. Subahan	**Reg. No.:**	192511121
Date:	28/10/26	**Duration:**	60 minutes

Code of Conduct

I T. Subahan *(192511121) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student:



Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. **Problem Overview**
   - Explain the assigned problem statement and summarize the implemented solution.
   - Describe the project objectives and key functionalities.
2. **Repository Organization**
   - Evaluate the repository structure, folder organization, and file naming conventions.
   - Justify how the repository layout supports maintainability and collaboration.
3. **Code Quality Review**
   - Review the source code for readability, modularity, coding standards, and documentation.
   - Identify strengths and areas for improvement.
4. **Version Control Evaluation**
   - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
   - Explain how version control supports collaborative software development.
5. **Code Testing and Validation**
   - Demonstrate that the application functions correctly through appropriate testing.
   - Present test cases, execution results, and screenshots as evidence.
6. **Repository Documentation**

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
- Suggest improvements to enhance usability and maintainability.

7. **Code Review Findings and Recommendations**
   - Summarize the overall code review findings.
   - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.

	organized.			
Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	**Mark**
Problem Analysis & Repository Organization(15%)	/4
Code Quality Review(25%)	/5
Version Control Evaluation(20%)	/4
Testing & Validation(15%)	/4
Documentation & Repository Maintenance(15%)	/4
Review Findings, Recommendations & Presentation(10%)	/4
TOTAL	/25

Vehicle Service and Maintenance Management System

1. Problem Overview

1.1 Problem Statement

The Vehicle Service and Maintenance Management System is a web-based application designed to simplify vehicle servicing activities. Vehicle owners often face difficulties booking service appointments, maintaining service records, remembering maintenance schedules, and tracking the progress of vehicle repairs.

Traditional methods depend on phone calls, paper records, and manual follow-ups. These methods can lead to misplaced records, missed maintenance schedules, and communication problems between vehicle owners and service centers.

The proposed system provides a centralized platform that enables vehicle owners to register their vehicles, book service appointments, view previous service records, receive maintenance reminders, and track service status.

A code review and repository evaluation are performed to assess the application's source code quality, repository structure, version control practices, testing procedures, and documentation.

1.2 Project Objectives

The main objectives are:

1. To develop a user-friendly vehicle service management application.
2. To provide online service appointment booking.
3. To maintain vehicle information and service history.
4. To help users identify upcoming maintenance requirements.
5. To track vehicle servicing progress.
6. To evaluate source code readability, modularity, and maintainability.
7. To assess Git usage, commit history, and branching practices.
8. To verify application functionality through testing.
9. To recommend improvements for future development.

1.3 Key Functionalities

S. No.	Functionality	Description
1	User Registration	Allows vehicle owners to create an account.
2	Vehicle Management	Stores vehicle details such as registration number, model, and manufacturer.
3	Appointment Booking	Allows users to select a service and request an appointment.
4	Service History	Maintains records of previously completed services.
5	Maintenance Reminders	Displays upcoming maintenance requirements.
6	Service Status Tracking	Displays whether the vehicle is booked, under service, or ready

		for collection.
7	Appointment Management	Allows users to view or cancel eligible appointments.

1.4 Implemented Solution Overview

The proposed solution uses a full-stack architecture consisting of a frontend, backend, and database.

- **Frontend:** HTML, CSS, and JavaScript provide the user interface.
- **Backend:** Node.js and Express process requests and implement business logic.
- **Database:** MySQL stores user, vehicle, appointment, and service-history information.
- **Version Control:** Git maintains source code history and supports collaborative development.
- **Containerization:** Docker provides a consistent application environment.

The application is divided into modules so that individual features can be reviewed, tested, and maintained independently.

Note: This is the proposed implementation for the assigned problem. Actual implementation details and review findings should be updated to match the source code in your repository.

2. Repository Organization

2.1 Proposed Repository Structure

A well-organized repository makes the source code easier to understand, test, review, and maintain.

The following structure is suitable for the Vehicle Service and Maintenance Management System.

```
vehicle-service-management/
│
├── frontend/
│   ├── index.html
│   ├── style.css
│   └── script.js
│
├── backend/
│   ├── server.js
│   ├── routes/
│   │   ├── vehicleRoutes.js
│   │   ├── appointmentRoutes.js
│   │   └── serviceRoutes.js
│   │
│   ├── controllers/
```

```
│   │   ├── vehicleController.js
│   │   ├── appointmentController.js
│   │   └── serviceController.js
│   │
│   ├── models/
│   │   ├── vehicleModel.js
│   │   ├── appointmentModel.js
│   │   └── serviceModel.js
│   │
│   ├── package.json
│   └── package-lock.json
│
├── database/
│   └── init.sql
│
├── tests/
│   ├── vehicle.test.js
│   └── appointment.test.js
│
├── screenshots/
│   ├── repository.png
│   ├── commit-history.png
│   └── test-results.png
│
├── Dockerfile
├── docker-compose.yml
├── .gitignore
├── .env.example
└── README.md
```

2.2 Purpose of Repository Components

Component	Purpose
frontend/	Contains the user interface and client-side scripts.
backend/	Contains server-side application logic.
routes/	Defines API endpoints for vehicles, appointments, and services.
controllers/	Handles incoming requests and prepares responses.
models/	Contains database access and data-related operations.
database/	Contains database initialization scripts.

tests/	Contains automated tests for application functionality.
screenshots/	Stores evidence for the assignment report.
Dockerfile	Defines how the application image is built.
docker-compose.yml	Defines the application and database services.
.gitignore	Excludes unnecessary files and local secrets from Git tracking.
.env.example	Documents the environment variables required by the application.
README.md	Explains the project, installation, configuration, and usage.

2.3 Evaluation of Repository Organization

The repository layout separates frontend code, backend logic, database scripts, and tests. This separation improves readability and allows developers to locate specific functionality quickly.

For example, appointment-booking logic can be maintained in the appointment route, controller, and model instead of being mixed with unrelated vehicle-management code.

The tests/ directory allows test cases to be maintained independently of application source code.

The README.md file provides a central location for project documentation.

The .gitignore file helps prevent temporary files, dependency folders, and local configuration files from being committed.

2.4 File Naming Conventions

The repository should follow consistent naming conventions.

- Use descriptive names such as appointmentController.js.
- Use consistent capitalization across files and folders.
- Use lowercase names for folders.
- Use meaningful test names such as appointment.test.js.
- Avoid vague names such as file1.js, newcode.js, or finalversion.js.

Evaluation: A consistent repository structure improves code navigation, reduces duplication, and supports collaborative development.

Evidence to include: A screenshot of the repository root and the expanded backend/, tests/, and database/ folders.

3. Code Quality Review

3.1 Purpose of Code Review

Code review is the systematic examination of source code to identify defects, improve readability, enforce coding standards, and make the application easier to maintain.

The review evaluates the following characteristics:

1. Readability
2. Modularity
3. Coding standards
4. Error handling
5. Input validation

6. Security
7. Maintainability
8. Testability

3.2 Code Readability

Readable code uses meaningful variable names, consistent indentation, and clear control structures.

Example of poorly readable code:

```
app.post("/book", (req, res) => {
  let d = req.body;
  if (d.date != "") {
    res.send("ok");
  }
});
```

Problems with this example include:

- The variable d does not clearly describe the request data.
- The endpoint name /book does not identify the resource.
- The date check does not validate the date format or whether it is in the future.
- The route does not provide a response when validation fails.
- The example does not demonstrate actual appointment storage.

Improved example:

```
app.post("/api/appointments", (req, res) => {
  const { vehicleId, serviceDate } = req.body;

  if (!vehicleId || !serviceDate) {
    return res.status(400).json({
      message: "Vehicle ID and service date are required."
    });
  }

  return res.status(201).json({
    message: "Appointment request validated successfully."
  });
});
```

The improved example uses descriptive variable names, a resource-oriented endpoint, and an explicit validation error response.

However, it only validates that the two fields are present. A production implementation must also validate their formats, verify that the vehicle exists and belongs to the authenticated user, check appointment availability, and persist the appointment in the database.

3.3 Modularity

Modularity means dividing an application into smaller components that perform specific responsibilities.

For this project, the backend can be divided into:

- **Routes:** Define HTTP endpoints.
- **Controllers:** Handle requests and responses.
- **Models:** Interact with the database.
- **Middleware:** Handle authentication, authorization, and validation.
- **Tests:** Verify the behavior of individual components.

For example, appointment creation should not require placing all database queries, validation rules, and response formatting inside one large server file.

Separating these responsibilities makes the application easier to test, debug, and extend.

3.4 Coding Standards

The following coding practices should be followed:

1. Use consistent indentation and formatting.
2. Choose meaningful variable and function names.
3. Avoid duplicated code.
4. Use const by default and let when reassignment is necessary.
5. Handle errors explicitly.
6. Validate user input on the server.
7. Use parameterized database queries.
8. Store configuration in environment variables.
9. Document complex business rules.
10. Keep functions focused on a single responsibility.

A code formatter and linter can help enforce consistent standards.

For a Node.js project, ESLint can identify many common coding problems, while Prettier can standardize formatting.

3.5 Error Handling

An application should handle invalid input and unexpected failures without crashing or exposing internal details.

For example:

```
app.get("/api/vehicles", async (req, res, next) => {
  try {
    // Retrieve vehicles from the database.
    const vehicles = [];

    return res.status(200).json(vehicles);
  } catch (error) {
    return next(error);
  }
```

});

This example demonstrates passing an error to Express error-handling middleware. The empty array is a placeholder, not a database implementation.

A complete application should retrieve records from the database and use centralized error-handling middleware to return appropriate HTTP responses.

3.6 Security Review

Security is important because the application may store user information, vehicle registration details, and service records.

The review should check:

- Whether user input is validated.
- Whether SQL queries use parameterized statements.
- Whether authentication and authorization are implemented.
- Whether passwords are securely hashed.
- Whether sensitive configuration is excluded from Git.
- Whether error responses expose internal information.
- Whether appointment records are restricted to authorized users.
- Whether dependencies are maintained and checked for vulnerabilities.

For example, database queries should avoid directly concatenating user input into SQL statements.

A parameterized query separates SQL instructions from supplied values and helps prevent SQL injection.

3.7 Strengths and Areas for Improvement

Review Area	Desired Strength	Possible Improvement
Readability	Meaningful names and consistent formatting	Introduce automated formatting.
Modularity	Separate routes, controllers, and models	Refactor oversized server files.
Validation	Reject invalid appointment details	Add schema-based validation.
Error handling	Return meaningful error responses	Implement centralized error middleware.
Security	Protect user and vehicle information	Add authentication and authorization tests.
Database access	Use structured database operations	Use parameterized queries and transactions where necessary.
Documentation	Explain complex business logic	Add API documentation.
Testing	Verify important application	Increase unit and integration test coverage.

behavior

Review conclusion: Code quality should be assessed using the actual source files. The examples above illustrate review criteria and possible improvements; they do not establish that these problems exist in your implementation.

4. Version Control Evaluation

4.1 Role of Git

Git is a distributed version control system that records changes to source code. It supports collaborative development by allowing multiple developers to work on separate branches and merge their contributions.

For the Vehicle Service and Maintenance Management System, Git can track changes to appointment booking, vehicle registration, service history, and deployment configuration.

4.2 Repository Evaluation Criteria

The following areas should be examined when evaluating the Git repository.

Evaluation Area	What to Check
Commit history	Whether changes are recorded consistently.
Commit messages	Whether messages clearly describe each change.
Branching	Whether features are developed on appropriate branches.
Merging	Whether branches are merged after review and testing.
Repository cleanliness	Whether unnecessary files are excluded.
Synchronization	Whether local changes are pushed to the remote repository.
Collaboration	Whether changes are reviewed and conflicts resolved appropriately.

4.3 Commit History Analysis

Display the repository's commit history using:

git log --oneline --graph --decorate --all

Example of meaningful commit messages:

feat: add vehicle registration
feat: implement appointment booking
feat: add service history
fix: validate service appointment date
test: add appointment validation tests
docs: update installation instructions
build: configure Docker Compose

These messages use a consistent format and describe the purpose of each change.

The example is illustrative. Include the commit history produced by your own repository in the final report.

4.4 Branching Strategy Evaluation

A suitable strategy uses:

- main for stable code.
- development for integrating features.
- feature/vehicle-registration for vehicle registration.
- feature/appointment-booking for appointment booking.
- feature/service-history for service history.

A feature can be merged into the development branch after testing and review. Once the integrated application is stable, the changes can be merged into main.

Example commands:

git switch -c development

git switch -c feature/appointment-booking

After implementing and committing the feature:

git add .

git commit -m "feat: implement appointment booking"

git switch development

git merge --no-ff feature/appointment-booking

4.5 Collaborative Development

Git supports collaboration in several ways:

1. Developers can work on separate features.
2. Changes can be reviewed before merging.
3. Conflicts can be identified and resolved.
4. Commit history provides traceability.
5. Remote repositories allow team members to synchronize work.
6. Rejected or incorrect changes can be investigated using earlier commits.

Pull requests can be used to review proposed changes before merging them into the main development branch.

Evaluation conclusion: A repository with meaningful commits, a clear branching strategy, documented setup instructions, and reviewed changes is easier to maintain and more suitable for collaborative software development.

5. Code Testing and Validation

5.1 Purpose of Testing

Testing determines whether the application satisfies its functional requirements and behaves correctly under expected and invalid conditions.

The Vehicle Service and Maintenance Management System should be tested for vehicle registration, appointment booking, service history, and service status tracking.

Testing should include both valid requests and invalid inputs.

5.2 Types of Testing

Unit Testing: Tests individual functions, such as appointment-date validation.

Integration Testing: Tests interactions between API routes, business logic, and the database.
Functional Testing: Verifies that features behave according to requirements.
Validation Testing: Checks whether the application rejects invalid or incomplete inputs.
Regression Testing: Ensures that new changes do not break previously working features.

5.3 Test Cases

Test ID	Test Scenario	Input or Action	Expected Result
TC01	Register a vehicle	Submit valid vehicle details	Vehicle is stored successfully.
TC02	Register a vehicle with missing details	Omit the registration number	Application returns a validation error.
TC03	Book a service appointment	Submit a valid vehicle ID and service date	Appointment is created.
TC04	Book an appointment without a date	Omit the service date	Application rejects the request.
TC05	View service history	Request history for an authorized vehicle	Previous service records are displayed.
TC06	Track service status	Request the status of an existing appointment	Current service status is displayed.
TC07	Access an unauthorized record	Request another user's vehicle record	Access is denied.
TC08	Restart the application	Restart containers and query saved data	Previously stored records remain available.

The expected result for TC07 assumes that authentication and record-level authorization are implemented.

5.4 Test Execution

If the backend uses Node.js's built-in test runner, tests can be executed using:

```
cd backend
node --test
```

The project must contain actual test files for this command to provide meaningful results.
To test the application manually, open the frontend in a browser, register a vehicle, book an appointment, and verify the resulting data.
For API testing, use an appropriate HTTP client or the browser's developer tools.

5.5 Test Result Documentation

Record the actual outcome of each test.

Test ID	Actual Result	Status
TC01	Fill in after execution	Pass/Fail

TC02 Fill in after execution Pass/Fail

TC03 Fill in after execution Pass/Fail

TC04 Fill in after execution Pass/Fail

TC05 Fill in after execution Pass/Fail

TC06 Fill in after execution Pass/Fail

TC07 Fill in after execution Pass/Fail

TC08 Fill in after execution Pass/Fail

A test should be marked **Pass** only when the observed result matches the expected result.

5.6 Testing Evidence

Include screenshots showing:

1. Successful application startup.
2. A valid vehicle registration.
3. A successful appointment booking.
4. Service history displayed in the application.
5. An appropriate error message for invalid input.
6. Test execution output.
7. Database records before and after an application restart.

Evaluation conclusion: Testing provides evidence that the system behaves as expected and helps detect defects before deployment.

6. Repository Documentation

6.1 Importance of Documentation

Documentation explains the purpose of the application, its architecture, installation procedure, configuration, and usage.

Clear documentation helps new developers understand the project without repeatedly asking the original developer for instructions.

6.2 README Evaluation

The repository should contain a README.md file with the following sections:

1. Project title and description.
2. Problem statement and objectives.
3. Main features.
4. Technology stack.
5. Repository structure.
6. Software prerequisites.
7. Installation instructions.
8. Environment configuration.
9. Instructions for running the application.
10. Testing commands.

11. API endpoint descriptions.
12. Screenshots.
13. Git branching and contribution guidelines.
14. Known limitations and future improvements.

6.3 Sample README Content

The following content can be used as the starting point for the project's README.

Project: Vehicle Service and Maintenance Management System

Description: A web-based application that helps vehicle owners manage service appointments, maintain service history, receive maintenance reminders, and track repair progress.

Technologies: HTML, CSS, JavaScript, Node.js, Express, MySQL, Git, and Docker.

Prerequisites: Git, Node.js and npm for local development, or Docker with Docker Compose for containerized execution.

Features:

- Vehicle registration and management.
- Service appointment booking.
- Service history.
- Maintenance reminders.
- Service status tracking.

Containerized execution:

git clone <your-repository-url>
cd vehicle-service-management
docker compose up --build -d
docker compose ps

Replace <your-repository-url> with the actual repository address.

Configure the required database passwords before starting the containers. The application's frontend and backend must be implemented and configured before the application can be accessed successfully.

To view logs:

docker compose logs -f backend

To stop the containers:

docker compose down

6.4 Documentation Evaluation

Documentation	Purpose	Recommended Improvement

Element		
README	Provides an overview of the project	Include complete setup instructions.
Installation Guide	Explains how to configure the application	Document prerequisites and environment variables.
Usage Guide	Explains how to use each feature	Include screenshots and example workflows.
API Documentation	Describes available endpoints	Document methods, parameters, and responses.
Testing Guide	Explains how to execute tests	Include commands and expected outcomes.
Contribution Guide	Defines development conventions	Explain branching and pull-request procedures.
Docker Guide	Explains container execution	Document environment setup and troubleshooting.

Evaluation conclusion: Complete documentation improves usability, reduces setup errors, and supports future development.

7. Code Review Findings and Recommendations

7.1 Summary of Review Findings

The review evaluates the project in five major areas:

1. Source code quality.
2. Repository organization.
3. Version control practices.
4. Testing and validation.
5. Documentation and maintainability.

The following table identifies desirable practices and possible improvements. These are review criteria and recommendations, not verified defects in an existing repository.

Area	Desired Practice	Recommended Improvement
Code quality	Readable and modular source code	Apply consistent formatting and separate responsibilities.
Repository organization	Clear folder structure	Keep frontend, backend, database, and tests organized.
Version control	Meaningful commits and branches	Review changes through pull requests.
Testing	Automated and manual test	Add unit and integration tests.

	coverage	
Documentation	Complete setup and usage instructions	Maintain a detailed README.
Security	Secure data handling	Add authorization checks and input validation.
Containerization	Reproducible deployment	Maintain Docker configuration and scan images for vulnerabilities.
Maintainability	Low duplication and clear responsibilities	Refactor repeated logic into reusable functions.

7.2 Code Quality Recommendations

- Use a consistent coding style.
- Separate routing, validation, database access, and business logic.
- Avoid duplicated functions.
- Validate all incoming data.
- Implement centralized error handling.
- Use parameterized SQL queries.
- Add comments only where they clarify non-obvious logic.
- Introduce automated linting and formatting.

7.3 Repository Management Recommendations

- Use descriptive commit messages.
- Develop features in separate branches.
- Review code before merging.
- Keep generated files and dependencies out of Git.
- Protect sensitive information from accidental commits.
- Maintain a clean repository structure.
- Tag stable releases when appropriate.

7.4 Testing Recommendations

- Add unit tests for validation and business rules.
- Add integration tests for database operations.
- Test invalid inputs and error conditions.
- Test authentication and authorization.
- Run regression tests after modifications.
- Automate test execution through a continuous integration workflow.

7.5 Documentation Recommendations

- Keep the README updated with the latest installation steps.
- Include screenshots of important application features.
- Document API endpoints and error responses.
- Provide environment variable examples.

- Explain Docker commands and common troubleshooting steps.
- Document the branching strategy and contribution procedure.

7.6 Future Enhancements

The system can be extended with the following features:

1. Email or SMS notifications for upcoming service appointments.
2. Online payment integration.
3. Service-center administrator dashboard.
4. Mechanic assignment and workload management.
5. Vehicle service cost estimation.
6. Automated deployment using CI/CD.
7. Application monitoring and centralized logging.
8. Enhanced role-based access control.
9. Automated database backups.
10. Performance optimization for large numbers of vehicles and appointments.

Conclusion

The Code Review and Repository Evaluation of the Vehicle Service and Maintenance Management System focuses on improving source code quality, repository organization, version control, testing, documentation, and maintainability.

A modular codebase makes the application easier to understand and extend. Git provides change tracking, branching, merging, and collaboration capabilities. Docker supports consistent execution across development and deployment environments.

Automated testing, clear documentation, meaningful commit messages, secure coding practices, and structured code reviews can further improve the reliability of the application.

By applying these practices, the Vehicle Service and Maintenance Management System can become easier to maintain, test, deploy, and extend in the future.